Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Lojik Devreler ve Tasarım

Bahar 2023

Dr. Öğr. Ü. İhsan Çiçek

8. Test Tezgahları ve Birleşik Lojik Yapıtaşları – Aritmetik Devreler

GEBZE TEKNİK ÜNİVERSİTESİ

1

Introduction

- Digital building blocks:
  - Gates, multiplexers, decoders, registers, arithmetic circuits, counters, memory arrays, logic arrays
- Building blocks demonstrate hierarchy, modularity, and regularity:
  - Hierarchy of simpler components
  - Well-defined interfaces and functions
  - Regular structure easily extends to different sizes

2

Arithmetic Circuits

- Digital building blocks:
  - Gates, multiplexers, decoders, registers, arithmetic circuits, counters, memory arrays, logic arrays
- Building blocks demonstrate hierarchy, modularity, and regularity:
  - Hierarchy of simpler components
  - Well-defined interfaces and functions
  - Regular structure easily extends to different sizes

3

Arithmetic Circuits

- Digital building blocks:
  - Gates, multiplexers, decoders, registers, arithmetic circuits, counters, memory arrays, logic arrays
- Building blocks demonstrate hierarchy, modularity, and regularity:
  - Hierarchy of simpler components
  - Well-defined interfaces and functions
  - Regular structure easily extends to different sizes

4

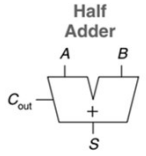
Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Arithmetic Circuits

- Digital building blocks:
  - Gates, multiplexers, decoders, registers, arithmetic circuits, counters, memory arrays, logic arrays
- Building blocks demonstrate hierarchy, modularity, and regularity:
  - Hierarchy of simpler components
  - Well-defined interfaces and functions
  - Regular structure easily extends to different sizes

Half-Adder

- An important operation for computers is to add numbers.
- For that, we will start with the most basic part of it which just adds **two bits** together and generates **a sum** and **a carry out**.
- Let's look at the truth table

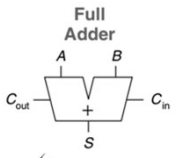


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$$S = A \oplus B$$
$$C_{out} = AB$$

Full-Adder

- Problem with the half-adder circuitry is that it does not account for the results of previous stages. (Carry in)
  - We cannot use the same circuit for summing up the second bits.
- Let's design a more complete adder having two bits and a carry in as inputs and sum and carry out as outputs.

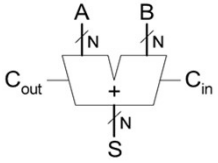


C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = AB + AC_{in} + BC_{in}$$

Multibit Adders (CPAs)

- N-bit adder sums **two N-bit inputs** as well as the **Cin** to produce **N-bit result S** and **Cout**.
- Common implementations of **carry propagate adders** (CPAs):
  - Ripple-carry (slow)
  - Carry-lookahead (fast)
  - Prefix (faster)

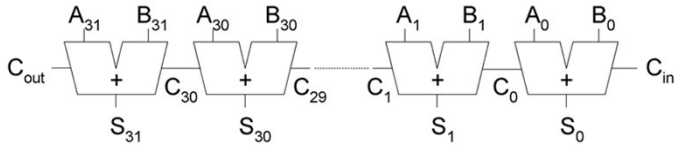


- Carry-lookahead and prefix adders faster for large adders but require more hardware

Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Ripple-Carry Adder

- Simplest way: Chain 1-bit adders together
- Carry ripples through entire chain
  - Each full adder needs to wait the result from the previous stage
- Not that good for large bit values, but good for simplicity. (same full adder block re-used multiple times)



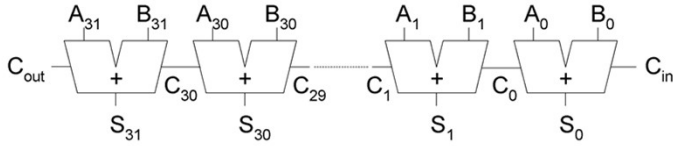
GEBZE TEKNİK ÜNİVERSİTESİ 9

9

Ripple-Carry Adder Delay

- If the delay of a single full adder is t_{FA}, the delay for the whole N-bit RCA will be:

$$\mathbf{t_{ripple} = N \times t_{FA}}$$



GEBZE TEKNİK ÜNİVERSİTESİ 10

10

Carry-Lookahead Adder

- The main reason RCA is slow is the need to calculate carry out for each bit so that it can be used in the next bit calculation.
  - You cannot start calculating $A_{31} + B_{31}$ before calculating all the others…
- We can take all the previous and results into the account when computing a specific bit. This type of adder is called **carry-lookahead adder**.
- The bit length can be divided into smaller blocks for **cl** calculation for efficiency.

GEBZE TEKNİK ÜNİVERSİTESİ 11

11

Carry-Lookahead Adder

- **generate (G)** and **propagate (P)** signals describe how a column or block determines the carry out.
- The **ith** column of an adder is said to **generate** a carry if it produces a carry out **independent** of the carry in.
  - when both A and B are 1. Hence

$$G_i = A_i B_i$$

- The ith column will **propagate** a carry if it produces a carry out whenever there is a carry in (i.e $C_{i-1} = 1$).
  - when either A or B are 1. Hence

$$P_i = A_i + B_i$$

GEBZE TEKNİK ÜNİVERSİTESİ 12

12

Carry-Lookahead Adder

- We can rewrite the carry logic for any given bit of the adder

$$G_i = A_iB_i$$
$$P_i = A_i + B_i$$

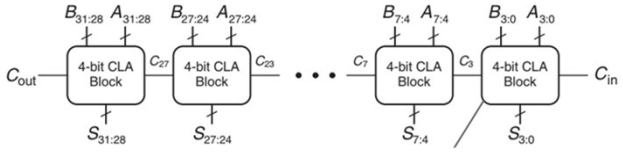
$$C_i = A_iB_i + (A_i + B_i)C_{i-1} = G_i + P_iC_{i-1}$$

- This way all the carries can be computed with combinational logic.
- For example - the carry for 3th bit is:

$$C_3 = G_3 + P_3(G_2 + P_2(G_1 + P_1C_0)$$

Carry-Lookahead Adder Blocks

- We can also use CLA as a block of let's say 4-bits, and apply the same calculation to blocks.
- A block is said to **generate** a carry if it produces a carry out independent of the carry in to the block.
- A block is said to **propagate** a carry if it produces a carry out whenever there is a carry in to the block.



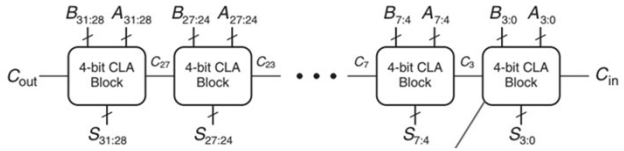
Carry-Lookahead Adder Blocks

- Using block generate and propagate signals, we can calculate carry out of the block. (Cj-1 is the carry in to the block)

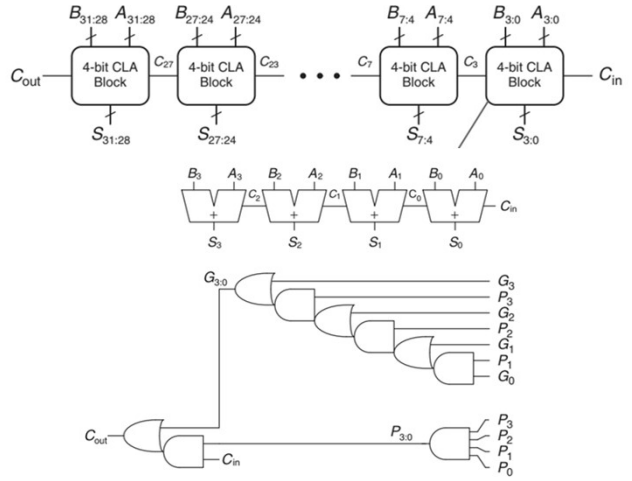
$$C_i = G_{i:j} + P_{i:j}C_{j-1}$$

- For example, for i=3 and j=0;

$$G_{3:0} = G_3 + P_3(G_2 + P_2(G_1 + P_1G_0)$$
$$P_{3:0} = P_3P_2P_1P_0$$



Carry-Lookahead Adder Block Delay



Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Carry-Lookahead Addition

- Step 1: Compute Gi and Pi for all columns
- Step 2: Compute G and P for k-bit blocks
- Step 3: Cin propagates through each k-bit propagate/generate logic (meanwhile computing sums)
- Step 4: Compute sum for most significant k-bit block

Carry-Lookahead Adder Block Delay

For N-bit adder divided into k-bit blocks, delay is

$$t_{CLA} = t_{pg} + t_{pg_block} + (N/k - 1)t_{AND_OR} + k \times t_{FA}$$

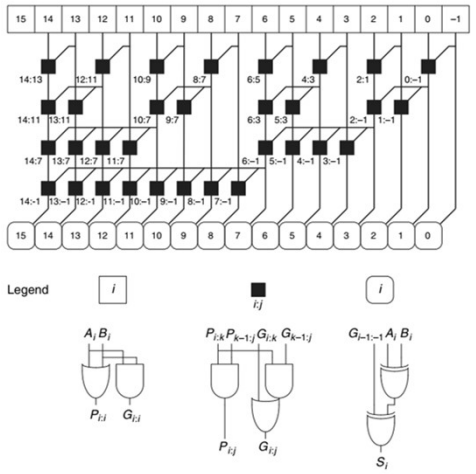
t_{pg} : delay to generate all P_i, G_i
t_{pg_block} : delay to generate all $P_{i:j}$, $G_{i:j}$
t_{AND_OR} : delay from C_{in} to C_{out} of final AND/OR gate in k-bit CLA block

An N-bit **carry-lookahead** adder is generally much **faster** than a **ripple-carry adder for N > 16**

Prefix Adder

- Prefix adders extend the generate and propagate logic to perform additions faster.
- First they compute G / P pairs for columns, then for blocks of 4, then for blocks of 8, 16 and so on.
- The sums are computed from these generate signals.

Prefix Adder Circuit



Prefix Adder Delay

- The critical path for an N-bit adder involves precomputation of Pi and Gi followed by log2N stages of prefix cells. Then there is the final XOR gate at the end.

$$t_{PA} = t_{pg} + \log_2 N(t_{pg_prefix}) + t_{XOR}$$

Addition Summary

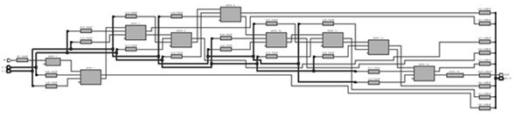
- It might be fine to use Ripple Carry Adder for a couple bits, but it will be beneficial to use alternatives for faster operation if the area is not that important
- Both **positive** and **negative** numbers can be added thanks to the 2's complement representations.
- There are other adder implementations
- In HDL design, the synthesis tools may pick the most appropriate design given the timing constraints.

Adder in SystemVerilog

```
module adder #(parameter N = 8)
              (input  logic [N-1:0] a, b,
               input  logic cin,
               output logic [N-1:0] s,
               output logic cout);

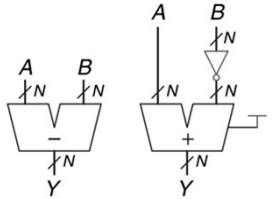
assign {cout, s} = a + b + cin;

endmodule
```



Subtraction

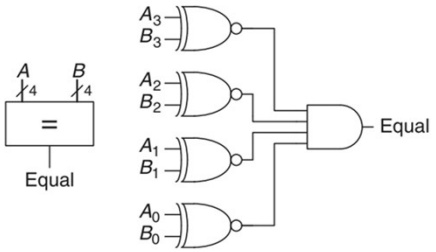
- Adders can be used to add negative numbers.
- Subtraction then becomes easy.
- Convert the second number to 2's complement form and sum both numbers
  - Add an inverter to the second input and feed in carry in as 1. (invert and +1)

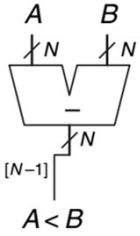


Comparators

A **comparator** determines whether **two** binary numbers are **equal** or if one is **greater** or **less than** the other.

- Equality comparator
- Magnitude comparator



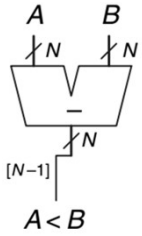


GEBZE TEKNİK ÜNİVERSİTESİ 25

25

Magnitude Comparator

- Magnitude comparison of signed numbers is usually done by computing **A - B** and looking at the **sign** bit. (MSB)
- If the result is **negative**, meaning the sign bit is 1, then **A is less than B**, if the result is **positive**, meaning the sign bit is 0, then **A is greater than or equal to B**.
- Need extra circuitry to detect overflow since that will not give the correct result



GEBZE TEKNİK ÜNİVERSİTESİ 26

26

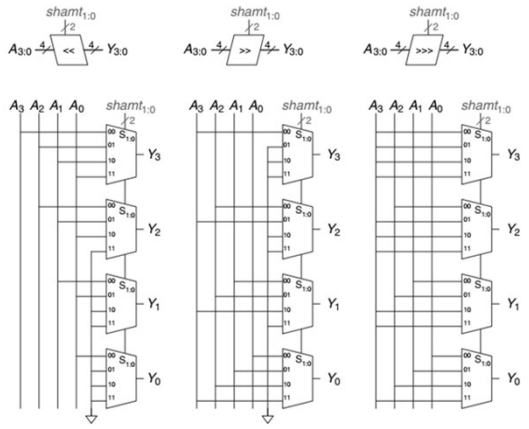
Shifters

- **Logical shifter:** shifts value to left or right and **fills** empty spaces with **0's**
  Ex: 11001 >> 2 =
  Ex: 11001 << 2 =
- **Arithmetic shifter:** same as logical shifter, but on right shift, **fills** empty spaces with the **old most significant bit** (msb)
  Ex: 11001 >>> 2 =
  Ex: 11001 <<< 2 =
- **Rotator:** rotates bits in a circle, such that bits shifted off one end are shifted into the other end
  Ex: 11001 ROR 2 =
  Ex: 11001 ROL 2 =

GEBZE TEKNİK ÜNİVERSİTESİ 27

27

Shifters



GEBZE TEKNİK ÜNİVERSİTESİ 28

28

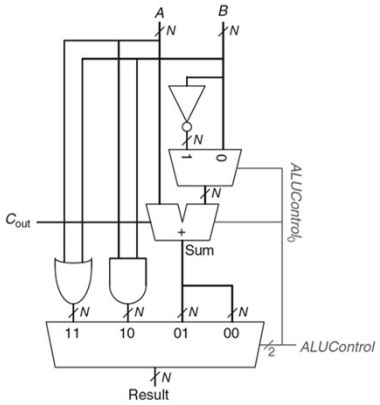
Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

ALU: Arithmetic Logic Unit

- An Arithmetic / Logic Unit (ALU) combines a variety of arithmetic and logical operations into a single unit.
- For example, a typical ALU might perform addition, subtraction, AND, OR operations.
- Each operation is encoded with a binary number.
  - Bit width of the binary numbers depend on the number of operations.
- The result is selected using a MUX with the encoded binary number
- ALU is the heart of most processors.

ALU: Arithmetic Logic Unit

$ALUControl_{1:0}$	Function
00	Add
01	Subtract
10	AND
11	OR



Example: Perform *A* + *B*

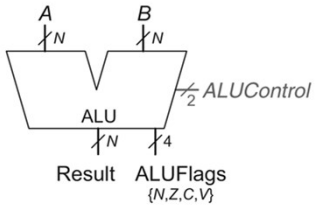
ALUControl = 00

$Result = A + B$

ALU with Status Flags

Some ALU implementations can produce extra output called flags

Flag	Description
N	*Result* is **N**egative
Z	*Result* is **Z**ero
C	Adder produces **C**arry out
V	Adder o**V**erflowed

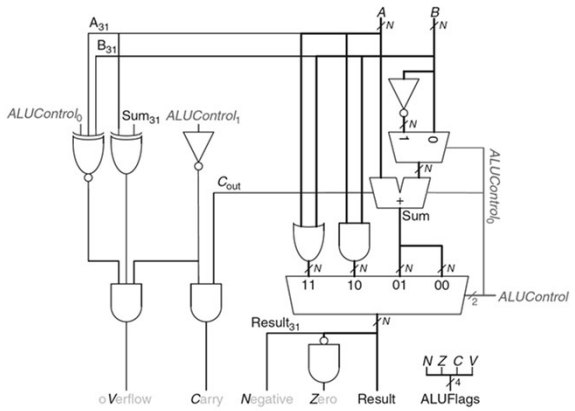


ALU with Status Flags

Some ALU implementations can produce extra output called flags:

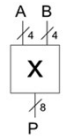
- **Negative** - When the sign bit is 1
- **Zero** - When all the bits are 0
- **Carry** - When there is a carry in the operation (in addition)
- **Overflow** - The addition of 2 same-signed numbers produces a result with the opposite sign

ALU with Status Flags Circuits



Multiplication

- Multiplication is a costly operation since it consist of many additions with chains.
- They work just like decimal multiplication.
- They are used to multiply and accumulate (MAC) operation which is an important operation for digital signal processing (DSP) algorithms.
- Sign and unsigned multiplications differ.

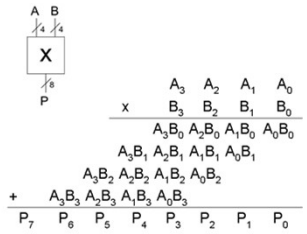


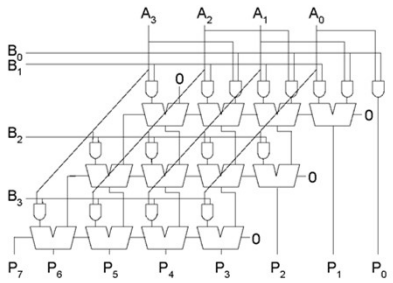
Decimal		Binary
230	multiplicand	0101
x 42	multiplier	x 0111
460	partial products	0101
+ 920		0101
9660		0101
		+ 0000
	result	0100011

230 x 42 = 9660 5 x 7 = 35

Multiplication circuit

- Just like adders, many different multiplier circuits exist with different speed / area trade-offs.
- In HDL design, the synthesis tools may pick the most appropriate design given the timing constraints.





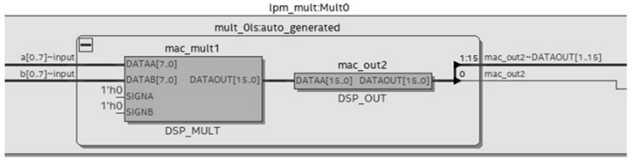
Multiplier in SystemVerilog

```
module mult #(parameter N = 8)
             (input  logic [N-1:0] a, b,
              output logic [2*N-1:0] s);

assign s = a * b;

endmodule
```





Öğrenci kopyasıdır. İzinsiz çoğaltılamaz ve dağıtılamaz.

Number Systems

Numbers we can represent using binary representations

- Positive numbers
  - Unsigned binary
- Negative numbers
  - Two's complement
  - Sign/magnitude numbers

What about **fractions**?

Fixed-Point Numbers

- There is a fixed point to divide the fractions and integers.
- Binary point is implied
- The number of integer and fraction bits must be agreed upon beforehand

For example: 6.75 using 4 integer bits and 4 fraction bits: 01101100

0110.1100

$2^2 + 2^1 + 2^{-1} + 2^{-2} = 6.75$

Floating-Point Numbers

- **Binary point floats to the right of the most significant 1**
- Similar to decimal scientific notation. For example, 273_{10} in scientific notation:

  $273 = 2.73 \times 10^2$
- In general, a number is written in scientific notation as:

  $\pm M \times B^E$

M = mantissa
B = base
E = exponent
In the example, M = 2.73, B = 10, and E = 2

FP Number Representation 1

Let's represent the value 228_{10} using a 32-bit floating point representation:

$228_{10} = 11100100_2$

If we rewrite it in scientific notation:

$= 1.11001 \times 2^7$



FP Number Representation 2

Let's represent the value 228_{10} using a 32-bit floating point representation in scientific notation:

$$228_{10} = \mathbf{1}.11001 \times 2^7$$

Since we can make it so that the first bit is always a 1, we can just omit using it to save one more bit:



FP Number Representation 3

- IEEE standardized representations in 1985 (**IEEE 754**)
- The exponent is biased to 127 (bias + exponent), which means if the exponent is 7, the bias exponent is 127+7=134 and if the exponent is -3, the biased exponent is -3+127 = 124

$$228_{10} = \mathbf{1}.11001 \times 2^7$$



Example

Write $\mathbf{-58.25_{10}}$ in floating point (IEEE 754)

Floating-Point: Special Cases

- The IEEE 754 has special cases to represent numbers such as zero, infinity, and illegal results (**N**ot **a** **N**umber)
- i.e. since there is an implicit 1, it is hard to represent number 0

Number	Sign	Exponent	Fraction
0	X	00000000	00000000000000000000000
∞	0	11111111	00000000000000000000000
- ∞	1	11111111	00000000000000000000000
NaN	X	11111111	non-zero

Floating-Point Precision

- Single-Precision:
  - 32-bit
  - 1 sign bit, 8 exponent bits, 23 fraction bits
  - bias = 127
  - roughly 7 significant decimal digit precision
- Double-Precision:
  - 64-bit
  - 1 sign bit, 11 exponent bits, 52 fraction bits
  - bias = 1023
  - roughly 15 significant decimal digit precision

Floating-Point: Rounding

- **Overflow** : number too large to be represented
- **Underflow** : number too small to be represented
- **Rounding modes:**
  - Down
  - Up
  - Toward zero
  - To nearest
- Example: round 1.100101 (1.578125) to only 3 fraction bits

Floating-Point Addition

1. Extract exponent and fraction bits
2. Prepend leading 1 to form mantissa
3. Compare exponents
4. Shift smaller mantissa if necessary
5. Add mantissas
6. Normalize mantissa and adjust exponent if necessary
7. Round result
8. Assemble exponent and fraction back into floating-point format

END OF LECTURE

Thanks